🚀 Async Python for Agentic AI: The Ultimate Cheatsheet

1. Async Ka Asli Matlab Kya Hai?

Agentic AI mein LLM aur APIs slow hote hain. Hum chahte hain ki hamara program (CPU) khali na baithe.

- **Synchronous (Dumb Waiter):** Normal `requests` library. Ek API ko call kiya, aur jab tak wahan se data nahi aaya, pura program **freeze** ho gaya.
- **Asynchronous (Smart Waiter):** `httpx` aur `asyncio`. Ek API ko call kiya, aur jab tak wo data la raha hai, CPU ne dusre tasks start kar diye.

---

2. The Two Superheroes: `await` vs `gather`

♂ The `await` Keyword (Line-by-Line)

Kaam: Apne current function ko wahin "pause" kar dena aur CPU ko bolna ki *"Main network ka wait kar raha hu, tu jaake dusre Agent/Task ki help kar de."* **Kab Use Karein:** Jab tasks ek dusre par **dependent** hon.

> **Example:** Pehle URL se text laao -> Fir LLM se summary banao. (Bina text ke summary nahi banegi).

♂ `asyncio.gather` (Ek Sath Daudna)

Kaam: Bahut saare independent tasks ko ek sath (parallel) fire karna.
Kab Use Karein: Jab tasks ek dusre se bilkul **independent** hon.

> **Example:** Agentic AI ko 3 alag Wikipedia pages padhne hain. Teeno ka aapas mein koi connection nahi hai, toh teeno ek sath load karo!

---

3. The "Dependency" Algorithm (Kab kya lagaye?)

Code likhne se pehle khud se ek sawaal pucho: **"Kya Step-2 ko start hone ke liye Step-1 ke data ki zaroorat hai?"**

- 🟢 **YES (Dependency hai):** Chup-chaap `await` lagao.
- 🟢 **NO (Independent hain):** Un sabhi tasks ko ek list mein daalo aur `asyncio.gather(*tasks)` maar do.

---

4. Copy-Paste Code Templates

📌 Case A: Dependent Tasks (The "await" flow)

Ek hi function ke andar, jab step 2 step 1 ka wait kare:

```
async def agent_task(url):
    # Step 1: Wait for data
    data = await http_client.get(url)

    # Step 2: Wait for LLM (depends on Step 1)
    llm_output = await groq_client.chat(data)

    return llm_output
```

📌 Case B: Independent Tasks (The "gather" flow)

Bahut saare URLs ya APIs ko ek sath marna:

```
async def main():
    urls = ["url1", "url2", "url3"]

    # 1. Tasks ki list banao (Yaha await mat lagana)
    tasks = [agent_task(url) for url in urls]

    # 2. Sabko ek sath fire kardo
    results = await asyncio.gather(*tasks)

    print(results) # 3 urls ka data ek sath aayega
```

---

5. 💡 Pro-Tips for Agents

1. **For-Loops ka Rule:** Agar tumhare code mein loop (`for x in y`) chal raha hai API call karne ke liye, toh pakka wahan `gather` lagega taaki loop ke tasks parallel ho jayein.
2. **`httpx.AsyncClient()`:** Hamesha isey `async with` (Context Manager) ke andar use karo. Taaki API call ke baad yeh khud close ho jaye aur memory leak na ho.
3. **Run Kaise Karein:** Async function normal tarike se call nahi hote. Code ke end mein hamesha `asyncio.run(main())` likhna padta hai (unless tum Jupyter/Colab mein ho, wahan sirf `await main()` chalta hai)